

TowerBot Operating Theatre

Simulation & Object Detection

Complete work log: Cursor AI-assisted development

Period: Early June 2026 | Generated: 2026-06-06

1. Executive Summary

This project set up TowerBot inside a custom Operating Theatre (OT) room in NVIDIA Isaac Sim 5.1 on an AWS EC2 instance (accessed via Amazon DCV). The goal was to configure robot sensors (front depth camera + LiDAR), publish data to ROS 2, and run YOLO object detection on hospital equipment using the custom Hospital.pt model (29 classes). Work progressed from an empty/broken scene to a working simulation where ROS topics publish live sensor data and YOLO detects multiple OT objects via an overview camera.

2. Environment & Infrastructure

Hardware & Access

- AWS EC2 GPU instance (NVIDIA Tesla T4, 12 GB VRAM)
- Remote desktop: Amazon DCV (DISPLAY=:0)
- Isaac Sim 5.1 via /home/ubuntu/isaac-env (Python 3.11)
- ROS 2 Jazzy at /opt/ros/jazzy

Important Rule Discovered

Use /usr/bin/python3 for ROS 2 and YOLO scripts. Do NOT use isaac-env Python for ROS/YOLO - it causes package conflicts and missing topics.

Repository & Key Paths

- Repo: /home/ubuntu/spotbot_ws/src/spotbot_lite
- OT stage: tower_bot_description/usd/finalot2.usd (layer over finalot.usd)
- Robot USD: tower_bot_description/usd/tower_bot_digital_twin_materials.usd
- YOLO model: tower_bot_perception/data/Hospital.pt (29 hospital classes)
- Isaac launcher: scripts/isaac-sim-dcv.sh
- Startup script: tower_bot_description/isaac/open_ot_light_stage.py

3. Project Goal (What You Asked For)

Starting request: run the robot in simulation, configure sensors and cameras, and detect objects in the OT environment - similar to real robot operation but in Isaac Sim.

- Build / load an OT room with hospital equipment
- Place TowerBot robot in the room at a useful pose
- Configure LiDAR and front depth camera with ROS 2 topics
- Enable robot movement via cmd_vel
- Run YOLO detection on every camera frame for all OT equipment classes
- Provide CEO-ready status updates throughout

4. Work Timeline (Top to Bottom)

Phase 1 - Initial Exploration & CEO Updates

- Started with goal: detect objects via robot sensors/cameras in simulation.
- Explored NVIDIA hospital environment images and Isaac Sim content browser.
- Early state: camera did not detect objects - project was at initial setup step.
- CEO asked about object detection model - clarified we use YOLO (Hospital.pt), not NanoOWL.
- Explained NanoOWL vs YOLO: NanoOWL is open-vocabulary; we use trained YOLO for hospital equipment.
- Defined task name: Isaac OT Sensor Configuration & Perception Validation.

Phase 2 - OT Environment Creation

- Robot was not visible; environment was empty at times.
- Built OT room stage with equipment placeholders (patient table, anesthesia machine, monitor, carts, etc.).
- Created USD stages: finalot.usd, finalot2.usd (user-saved layout layer).
- Placed TowerBot beside operating table at pose (-0.2, 1.7, 0.1, yaw 180 degrees).
- Fixed red-colour robot material issues and missing mesh references.

Phase 3 - Sensor Configuration (Major Effort)

- Created setup_isaac_ot_sensors.py - wires RTX LiDAR + camera to ROS 2 ActionGraph.
- ROS topics configured to match real robot: /scan, /front_depth/rgb0/image, /front_depth/depth0/image_raw.
- Isaac Script Editor used for in-GUI fixes (configure_sensors_PASTE.py and related scripts).
- Issues: ROS 2 OmniGraph menu options missing, Script Editor paste errors, extension load timing.
- Solution: automated sensor setup in open_ot_light_stage.py startup + Script Editor paste scripts.
- Verified topics with: ros2 topic list (after sourcing ROS + spotbot_ws install).

Phase 4 - Robot Visibility & Physics Issues

- Repeated issue: tower_bot visible in Stage tree but mesh missing (empty Xform).
- Robot moved too fast and collapsed when dragged in viewport (PhysX / articulation issues).
- On Stop/Play, robot jumped back to door pose - fixed with lock_spawn_and_save_PASTE.py.
- Created robot_spawn_utils.py with spawn_visible_robot() - adds USD reference without flattening.
- Deleted and re-spawned broken tower_bot multiple times via restore scripts.
- Created fix_towerbot.sh for full automated restart with respawn + sensor rewire.

Phase 5 - Isaac Sim Stability on DCV

- Isaac Sim not opening inside DCV / hanging on load.
- Multiple Isaac instances caused conflicts - fixed with pkill + fresh_start_isaac.sh.
- DCV terminal issues - PATH polluted by isaac-env; fixed by using /usr/bin/python3.
- Isaac window switching / focus issues on DCV - use Alt+Tab, single instance only.
- Checked logs at ~/.nvidia-omniverse/logs/isaac-sim-dcv/ for load failures.

Phase 6 - YOLO Object Detection

- Installed ultralytics; ran ot_yolo_test.py against /front_depth/rgb0/image.
- Fixed: tower_bot_perception package not found - needed source spotbot_ws/install/setup.bash.
- Expanded from single-image detection to continuous multi-frame detection.
- Created overview camera (ot_yolo_overview_cam) for bird's-eye view of full OT room.
- Fixed black/half camera frames - quaternion bug (use AddTransformOp not separate OrientOp).

TowerBot OT Environment - Isaac Sim + ROS2 + YOLO

- Auto-save detections: ~/ot_yolo_latest.jpg, ~/ot_detections/frame_*.jpg.
- Detected classes (latest): Monitor, Pillow, C-Arm Machine, Rack, C-Arm Monitor, Patient table.
- 29-class model limited by Isaac box meshes vs real hospital training photos.

Phase 7 - Scene Preservation & finalot2.usd

- User restored scene to finalot2.usd with bedside robot pose.
- TOWER_BOT_PRESERVE_SCENE=1 keeps equipment + robot pose on restart.
- Root cause found: finalot2.usd saves empty tower_bot Xform (pose only, no mesh reference).
- Added auto-save after robot respawn so mesh reference persists across reloads.
- RESTORE_MY_SCENE.py on Desktop - one-click robot restore + auto-save.

5. Issues Faced & How They Were Resolved

Robot not visible (most frequent)

Cause: finalot2.usd stores robot position only, not the USD mesh reference. Stage shows tower_bot Xform with yellow gizmo but no body. Fix: spawn_visible_robot() on startup + RESTORE_MY_SCENE.py + fix_towerbot.sh. Do not File->Save when tower_bot has no children in Stage panel.

Robot collapses / moves too fast

Cause: broken PhysX articulation, invalid transforms, or dragging in Play mode. Fix: respawn fresh robot reference, pause sim before moving, use cmd_vel for motion.

Robot jumps to door on Stop

Cause: spawn pose saved in wrong layer. Fix: lock_spawn_and_save_PASTE.py saves correct bedside pose to stage.

No ROS topics (/scan, /front_depth)

Cause: Isaac not in Play mode, sensors not wired, or wrong Python environment. Fix: Press Play, run setup_isaac_ot_sensors, use /usr/bin/python3, ROS_DOMAIN_ID=0.

Camera black or half-black frames

Cause: wrong camera quaternion / orient op on overview cam. Fix: AddTransformOp with rotation matrix in ot_yolo_scene_setup.py.

YOLO detects only Stand or one class

Cause: using robot front camera (narrow view) instead of overview camera. Fix: rewire ROS camera to /World/ot_yolo_overview_cam.

tower_bot_perception not found

Cause: missing source install/setup.bash. Fix: source /home/ubuntu/spotbot_ws/install/setup.bash before ros2 run.

Isaac Sim not opening / hanging

Cause: zombie isaacsim processes, GPU memory, or extension load timeout. Fix: pkill isaacsim, wait, relaunch via isaac-sim-dcv.sh or fix_towerbot.sh.

Script Editor paste / copy not working in DCV

Cause: DCV clipboard limitations. Fix: open scripts from Desktop (RESTORE_MY_SCENE.py) via File->Open in Script Editor.

HUB NOT DETECTED in Isaac UI

Omniverse hub warning - generally unrelated to simulation; can be ignored for local EC2 work.

6. Key Files & Scripts Created

- open_ot_light_stage.py: Isaac startup: open stage, respawn robot, wire sensors, overview cam.
- robot_spawn_utils.py: spawn_visible_robot(), robot_needs_respawn(), persist_stage_with_robot().
- setup_isaac_ot_sensors.py: LiDAR + camera ROS 2 ActionGraph wiring.
- ot_yolo_scene_setup.py: Overview camera + OT equipment positioning for YOLO.
- cmd_vel_graph.py: ROS /TowerBot/Teleop/cmd_vel bridge for robot movement.
- ot_yolo_test.py: Live YOLO on camera topic; saves annotated images.
- ot_yolo_capture_all.sh: One-shot multi-frame YOLO capture.
- save_raw_camera_frame.py: Save raw camera JPEG for verification.
- isaac-sim-dcv.sh: Launch Isaac Sim in DCV with startup script.
- fix_towerbot.sh: Full restart: kill Isaac, respawn robot, wait for topics.
- RESTORE_MY_SCENE.py (Desktop): Script Editor: restore robot + sensors + auto-save.
- SHOW_ROBOT.py (Desktop): Script Editor: force robot respawn.
- CHECK_YOLO_KEEP_SCENE.py (Desktop): Wire sensors without moving robot/equipment.

7. Object Detection - What to Tell CEO

We use YOLOv8 segmentation with a custom Hospital.pt model trained on 29 hospital/OT equipment classes (patient table, anesthesia machine, monitor, trolley, etc.). This is NOT NanoOWL. The same perception stack runs in Isaac Sim (on ROS camera topics) and on the physical robot. NanoOWL is an alternative open-vocabulary detector in Isaac; we chose YOLO because we have a trained hospital-specific model.

8. Current Status (End of Session)

Working

- Isaac OT sim loads finalot2.usd on AWS EC2 via DCV
- ROS 2 topics when Play is LIVE: /scan, /front_depth/rgb0/image, /front_depth/depth0/image_raw
- YOLO live detection via overview camera - 6+ equipment classes detected
- Images saved: ~/ot_yolo_latest.jpg, ~/ot_detections/frame_*.jpg
- Robot respawn + auto-save on startup (latest code change)

Still Pending

- Robot mesh may disappear if finalot2.usd saved without mesh children
- LiDAR invalid prim path warning: /World/tower_bot/lidar_link/.../rtx_lidar
- Full 29-class detection - limited by simple box meshes in sim vs real photos
- Realtime factor benchmarking (CEO asked - comparable to Gazebo RTF metric)

9. Command Reference

Restart Isaac + fix robot

```
bash /home/ubuntu/fix_towerbot.sh
```

Live YOLO detection

```
source /opt/ros/jazzy/setup.bash
export ROS_DOMAIN_ID=0
export PATH=/usr/bin:/bin:/opt/ros/jazzy/bin:$PATH
/usr/bin/python3 ../scripts/ot_yolo_test.py
```

One-shot YOLO capture

```
bash ../scripts/ot_yolo_capture_all.sh
```

Check ROS topics

```
ros2 topic list | grep -E 'scan|front_depth'
```

Save camera test image

```
/usr/bin/python3 ../scripts/save_raw_camera_frame.py ~/ot_test.jpg
```

View latest detection

```
eog ~/ot_yolo_latest.jpg
```

Isaac logs

```
ls -lt ~/.nvidia-omniverse/logs/isaac-sim-dcv/
```

10. End-to-End Workflow

1. Launch Isaac via fix_towerbot.sh or isaac-sim-dcv.sh
2. open_ot_light_stage.py runs automatically:
 - a. Opens finalot2.usd
 - b. Checks if tower_bot has mesh; respawns if empty
 - c. Wires LiDAR + camera ROS 2 graphs
 - d. Creates overview camera for YOLO
 - e. Auto-saves stage with robot mesh
3. Isaac Play starts (TOWER_BOT_AUTO_PLAY=1)
4. Terminal: run ot_yolo_test.py with system Python
5. YOLO subscribes to /front_depth/rgb0/image (overview cam)
6. Detections printed + saved to ~/ot_yolo_latest.jpg

11. CEO Summary (One Paragraph)

We built and stabilized a TowerBot Operating Theatre simulation on AWS using NVIDIA Isaac Sim. The robot is placed in a custom OT room with hospital equipment. Sensors (LiDAR and front camera) publish live ROS 2 data, and our custom YOLO Hospital model detects OT equipment from a room overview camera. Main challenges were robot mesh disappearing on scene save, Isaac/DCV stability, and camera wiring - all addressed with automated startup scripts.

YOLO currently detects 6+ equipment classes live; expanding to all 29 classes requires richer 3D equipment models in simulation.

12. Suggested Task Names for Tracking

- Isaac OT Sensor Configuration & ROS 2 Integration
- TowerBot OT Scene Setup & Digital Twin Placement
- YOLO Perception Validation in Simulated OT Environment
- Isaac Sim Stability & DCV Deployment Hardening
- Sim-to-Real Perception Parity (Isaac + Physical Robot)